

Demonstration of TD-error movement in an MLP

Alex Zhang

University of California, Berkeley

I. INTRODUCTION

Reinforcement learning is a type of machine learning where agents are given reward based on their performance on a task. This type of learning has enabled many advances in AI like Google Deepmind’s AlphaGo, and is especially relevant today as data becomes scarce, as reinforcement learning does not require the same data that standard supervised learning does. There is a specific type of reinforcement learning that uses temporal difference error to update weights (the prediction of the reward received vs actual reward received). This is especially biologically plausible, as Schultz et al. (1997) showed that monkeys display this kind of learning by measuring dopamine neurons firing in relation to a cue that signified a reward was coming [1]. Specifically, they showed that as the monkeys learned to associate the cue with the reward, their dopamine neurons fired at the cue instead of when the reward was given (where it fired originally, before the monkeys learned). In this experiment, the behavior of an error spike at the reward time-step moving to the cue time-step is replicated using a standard multilayer perceptron trained using backpropagated TD-learning, effectively replicating Schultz et al.’s experiment with artificial neural networks.

II. METHOD

A. Failed Trials

My experimentation began with the reinforcement learning task Cartpole, as my previous research in TD-based reinforcement learning was heavily involved with this task [2], [3]. However, quickly after I implemented a learner on Cartpole, I realized that it may not have been the best choice to demonstrate the experiment I intended to. Though this was a stateful environment, the reward is given for time in the air, specifically 1 per second. It would have been a larger hassle to define cues and reward delays. I leave this for future work, though, as with more time, an actor-critic network definitely can demonstrate the same TD-error movement. This would be the next thing to pursue after this current project.

I next switched to a simpler environment. I implemented a cue-reward task with a tabular TD(0) learner over a lookup table indexed by time since cue. While this reproduced the three target phenomena cleanly, I wanted a more natural learning method, as this was deterministic and seemed almost like cheating to me.

B. MLP

Thus, I pivoted to using a standard multilayer perceptron (MLP). I constructed a simple, fully connected neural net with two hidden layers (32 neurons each), each connected to tanh()

activation functions, all feeding into a linear readout layer that predicts the state. The TD-error δ_t is defined recursively by $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$, where r_{t+1} is the reward given, γ is the discount factor, $V(s_{t+1})$ is the model’s prediction based on the next state, and $V(s_t)$ is the prediction based on the current state. Given that the model made a perfect prediction, $V(s_t) = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$ and $\gamma V(s_{t+1}) = \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots$ are true, meaning $\gamma V(s_{t+1}) - V(s_t) = -R_{t+1}$, and subsequently, $\delta_t = 0$. Thus, δ_t is a good measure on how wrong the model is in its prediction of state, as a perfect prediction would lead to zero error, and error scales up as the prediction becomes worse. I fed that error into standard backpropagation using MSE, where loss is defined:

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^T \delta_t^2$$

In the environment, the delay between the cue and reward was a fixed value, while the cue’s timestep was chosen randomly for each trial to ensure the model didn’t generalize based on timing and actually learned the cue.

This network’s TD error was first measured untrained; then, it was trained for 150,000 epochs and measured again. Using the trained model, an ablation was conducted where the reward was withheld from the model and TD error was recorded. This pipeline was conducted across 5 random seeds. Claude code and Qwen 3.8 27b were used to assist in coding.

III. RESULTS

As seen in Figure 1 in Appendix A, in the untrained model, error spikes at the reward, as the prediction of the model is contradicted heavily by the actual reward. After training, the error spikes at the cue. At the cue, the model of course doesn’t expect the cue coming, so some error spikes (though considerably less than the untrained model’s spike at reward). However, the trained model fully expects the reward after the cue, so there is no spike there. In the ablation, the model does not get the reward it expected. Thus, the $V(s_t)$ blows up as it predicts the reward. Since r_{t+1} is still zero this time, TD error goes strongly negative.

IV. CONCLUSION

This writeup demonstrated that Schultz et al.’s experiment on monkeys can be replicated using artificial neural networks, inviting more research to be done on the intersection between modern AI and neuroscience.

REFERENCES

- [1] W. Schultz, P. Dayan, and P. R. Montague, "A neural substrate of prediction and reward," *Science*, vol. 275, no. 5306, pp. 1593–1599, 1997.
- [2] A. Zhang, "ACE: ACE: A Novel and Biologically Inspired SNN Neuromodulation Training Rule," under review for ICLR.
- [3] A. Zhang, "Morphing with Spikes" under review for ICLR.

V. APPENDIX A

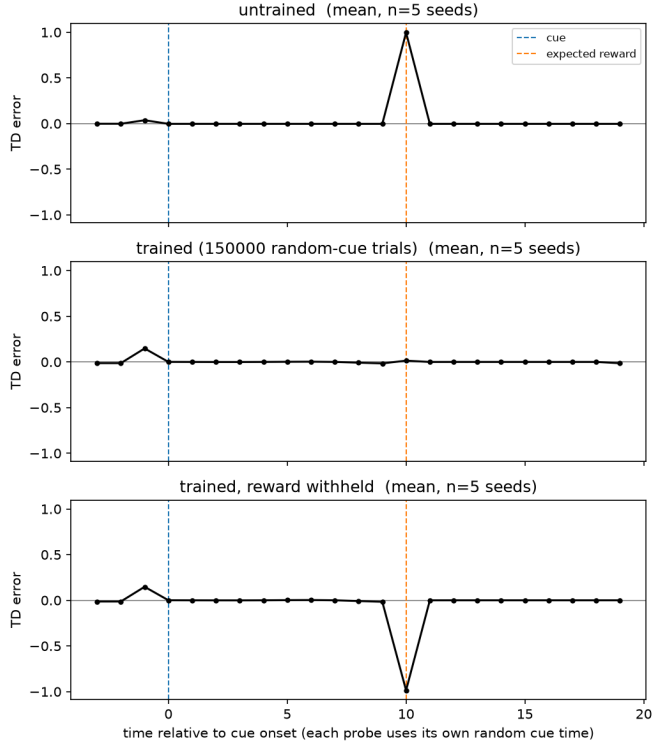


Fig. 1. TD error as a function of time relative to cue onset, averaged across 5 random seeds. Top: untrained model. Middle: model after 150,000 training trials. Bottom: trained model with reward withheld.